Architecture Decision Record

Federated Edge-Adapter Architecture for the Smart Disaster Resource Exchange Platform

1. Context

Large-scale disasters expose a recurring coordination gap: verified relief organisations — hospitals, fire departments, NGOs — each hold inventories of critical resources but have no shared, trustworthy channel through which to publish availability and discover needs across organisations. Coordinators fall back on phone calls and spreadsheets precisely when speed and accuracy matter most.

Three constraints shape the design. Participation must be restricted to verified organisations, favouring organisation-level onboarding and authentication over open self-service signup. Resource owners must retain approval authority, requiring an explicit request-approval workflow rather than a marketplace. Most consequentially, the platform must be designed for failure: disaster-zone connectivity is unreliable by definition, and an organisation's system going dark for hours before reconnecting is a normal case, not an edge case. Organisations also cannot be asked to replace or heavily modify existing internal systems, whether mature hospital IT or a volunteer group with no IT capability at all.

These constraints exclude a centralized monolith with direct database access to each participant, and instead call for a distributed, event-driven architecture in which each organisation behaves as an autonomous node that may disappear and reappear at any time.

2. Architectural Goals

- **Reliability/partition tolerance:** Organisations keep operating locally when disconnected, and reconcile correctly on reconnect.
- **Interoperability:** Heterogeneous legacy systems are normalised at the boundary rather than forced onto one internal data model.
- **Availability:** Near-real-time discovery favours event-driven propagation over batch sync or polling.
- **Security/trust:** Every event is attributable to an authenticated, verified organisation that controls what it exposes externally.
- **Scalability:** Bursty load — a reconnecting organisation flushing hours of queued updates — must not cascade into failure.
- **Extensibility:** Event bus lets new organisation types and resource categories be added without coordinated redeployment.

3. Key Architectural Decisions

3.1 Federated architecture with an Org Edge Adapter

A centralised design reaching into each organisation's database was rejected: it violates the closed-system constraint and has no answer for unreachable participants. Instead, a lightweight Edge Adapter runs inside each organisation's own infrastructure as the sole boundary across which data crosses outward — the organisation, not the platform, decides what is published. This applies the Anti-Corruption Layer pattern, translating legacy conventions to the platform's standard schema at the edge, at the cost of a one-time field-mapping configuration per organisation.

3.2 Three-tier integration model

Because organisations differ in technical maturity, a single mandatory API contract would be too demanding for volunteer groups and too primitive for hospitals. The adopted model offers push (webhook), polling (legacy database or CSV diff), and manual (secured web form) tiers, all converging on the same standardized event before reaching central services — satisfying low-friction integration without lowering data quality.

3.3 Local outbox queue and asynchronous forwarding

A synchronous design, where the organisation's system waits on an HTTP call to the central platform, was rejected: a severed connection would hang the request and block a real-world action behind an unrelated network failure. The adopted Transactional Outbox pattern writes every change to a local embedded database first and forwards asynchronously, so operations never block on central availability, at the accepted cost that the platform's view of an organisation's resources is not guaranteed current.

3.4 API Gateway as the synchronous-to-asynchronous bridge

A single ingress point performs authentication, rate limiting, and schema validation, chosen over exposing internal services directly, since one well-defended boundary is easier to secure and reason about. The gateway holds no datastore connection; accepted requests become bus events, and only a confirmed publish triggers acknowledgment,

permitting safe deletion from the adapter's outbox. Rate limiting also absorbs the burst of a reconnecting adapter flushing its backlog.

3.5 Event-driven microservices over an event bus

A monolith with synchronous inter-service calls was rejected because it propagates failure across the system. Independent microservices — registry, catalog, workflow, discovery/matching, reconciliation, audit, notification — instead communicate only via a persistent, replayable event log. A crashed service recovers by replaying missed events, and read-optimised views such as the resource catalog are built as projections of the stream — a lightweight CQRS separation of writes from reads.

3.6 Staleness-aware discovery and dual consistency

Rather than treating all listings as equally live, the registry derives an online/stale/offline status from heartbeats, which the catalog attaches to records and discovery surfaces explicitly to coordinators, preserving human judgement over hidden uncertainty. This pairs with a deliberate asymmetry: a request's own lifecycle is kept strongly consistent via ACID transactions, since claim conflicts must be prevented outright, while an organisation's real-world resource state is allowed to be eventually consistent, as offline tolerance demands.

3.7 Reconciliation and human-in-the-loop conflicts

An organisation may allocate a resource locally during an outage that the platform has already matched to another request. Silent auto-resolution was rejected as too dangerous. A reconciliation service instead replays a reconnected adapter's outbox with idempotency keys for exactly-once processing, detects any conflict, and routes only that conflict to a human coordinator for manual re-matching.

3.8 Polyglot persistence

No single store fits every access pattern: the catalog needs flexible, versioned documents; workflow needs strict transactions; the bus needs an append-only log for replay; and audit needs immutable, write-once storage. Each store is matched to its owning service's needs, at the accepted cost of more components to deploy and monitor.

3.9 Supporting monitoring and identity capabilities

Independent outage detection, distributed tracing, and centralised identity enforcement are layered onto the gateway and service mesh to keep the partition-tolerant, event-driven design observable and enforceable: detection corroborates heartbeat-derived staleness, tracing follows a request across decoupled asynchronous services, and identity enforcement gives the gateway one place to verify an event's origin.

4. Architectural Tradeoffs

- **Eventual consistency by default:** An honest representation of hours-long disconnections, accepted so coordinators see staleness rather than false certainty.
- **Operational complexity:** seven decoupled services over a shared bus cost more to operate than a monolith — the price of decoupling and partition tolerance.
- **Monitoring overhead:** polyglot persistence and per-organisation adapters multiply components requiring observability, offset by budgeted tracing and outage detection.
- **Onboarding cost:** the Anti-Corruption Layer needs one-time field mapping per organisation, in exchange for never altering that organisation's systems again.
- **Deferred resolution:** routing conflicts to a human trades automation for reduced risk of an unattended, incorrect allocation.

5. Expected Outcomes

The federated edge-adapter architecture delivers multi-agency interoperability by letting each organisation integrate at whatever tier it supports, while central services see only a standardised schema. Reliable coordination follows from separating the strongly consistent workflow state machine from the eventually consistent, staleness-annotated catalog. Resilience during disasters comes primarily from the local outbox, which decouples operations from internet availability, and the reconciliation service, which recovers safely from partitions rather than merely detecting them. Secure communication follows from the edge adapter as the sole authenticated boundary, with the gateway enforcing authentication, validation, and rate limiting before any event reaches the bus. Future scalability is supported by the decoupled, event-driven topology, which admits new resource types, organisation categories, or event consumers without altering organisations already integrated at the edge.